PYTHON ACTIVITY-BASED LEARNING

Consolidated report on 10 practical Python case studies

Submitted by: Deepak

Roll Number: 26000931

Branch: CSM | Section: 2

First-Year Engineering

This report brings together ten small Python programs based on everyday calculations and decision-making problems. Each activity includes its aim, rules, program logic, sample code, and a recorded output.

Contents

1. University Scholarship Allocation

2. Progressive Electricity Billing

3. Hospital Billing and Patient Classification

4. Employee Payroll and Deduction Analysis

5. Multi-Plan Telecommunications Cost Optimization

6. Railway Fare and Reservation Calculator

7. Retail Billing with Multiple Discount Rules

8. Manufacturing Cost and Profitability Analysis

9. Multi-Currency Travel Budget Analysis

10. Campus Event Financial Planning

Note: The rates, limits, and policies used in these programs are classroom examples, not official tariffs or regulations.

Activity 1: University Scholarship Allocation

What the program does

Evaluate students using marks, attendance and family income; rank eligible students with clear tie-breaking rules.

Rules used in this activity

All subject marks are out of 100; minimum subject mark 40, average 70, attendance 75%, income at most Rs 300000. Ties: higher average, then attendance, then lower income, then name.

How I approached it

I calculated the average and checked each eligibility condition. I then sorted only the eligible students using the given tie-breaking order.

Python program

```
students = [
       {"name":"Asha", "marks":[85, 79, 90], "attendance":92, "income":180000},
       {"name":"Bala", "marks":[88, 76, 90], "attendance":90, "income":160000},
       {"name":"Charan", "marks":[92, 35, 95], "attendance":94, "income":120000},
       {"name":"Divya", "marks":[80, 82, 92], "attendance":92, "income":150000}
   ]
   eligible = []
   for student in students:
       average = sum(student["marks"]) / len(student["marks"])
       if min(student["marks"]) >= 40 and average >= 70 and student["attendance"] >= 75
and student["income"] <=
   300000:
           student["average"] = average
           eligible.append(student)
   eligible.sort(key=lambda s: (-s["average"], -s["attendance"], s["income"], s["name"]))
   print("SCHOLARSHIP RANKING")
   for rank, student in enumerate(eligible, 1):
       print(rank, student["name"], "Average:", round(student["average"], 2))
   print("Eligible:", len(eligible), "of", len(students))
```

Activity 1: Results and observations

Sample execution

```
Terminal - case_01.py

$ python programs/case_01.py
SCHOLARSHIP RANKING
1 Divya Average: 84.67
2 Asha Average: 84.67
3 Bala Average: 84.67
Eligible: 3 of 4
```

Conclusion

The program demonstrates the use of Python variables, arithmetic, conditions, loops and data structures to solve

Testing and observations

For the sample values, asha ranked first; Charan excluded because one subject is below 40.

I can also test the program by changing a boundary value, such as a minimum mark, consumption slab, or budget. Trying zero or invalid values helps check how the conditions behave.

What I learned

This activity helped me practise Python variables, calculations, conditional statements, loops, and working with lists or dictionaries where needed.

Activity 2: Progressive Electricity Billing

What the program does

Compute monthly slab-wise electricity bills, compare months and flag unusual increases.

Rules used in this activity

First 100 units Rs 2/unit, next 100 Rs 3/unit, remaining Rs 5/unit; fixed Rs 50; surcharge 5% if units exceed 300; rebate 5% for units at most 100; unusual rise above 30% month-on-month.

How I approached it

I divided the electricity units into slabs, added fixed charges, and applied the rebate or surcharge where needed. I compared each month with the previous bill.

Python program

```
units_by_month = {"Jan":90, "Feb":170, "Mar":360, "Apr":200}
   previous = None
   for month, units in units_by_month.items():
       charge = min(units, 100) * 2
       charge += min(max(units - 100, 0), 100) * 3
       charge += max(units - 200, 0) * 5
       total = charge + 50
       if units > 300:
           total *= 1.05
       if units <= 100:
           total *= 0.95
       warning = previous is not None and total > previous * 1.30
       print(month, "Units:", units, "Bill: Rs", round(total, 2), "Unusual rise:", warning)
       previous = total
```

Activity 2: Results and observations

Sample execution

```
Terminal - case_02.py

$ python programs/case_02.py
Jan Units: 90 Bill: Rs 218.5 Unusual rise: False
Feb Units: 170 Bill: Rs 460 Unusual rise: True
Mar Units: 360 Bill: Rs 1417.5 Unusual rise: True
Apr Units: 200 Bill: Rs 550 Unusual rise: False
```

Conclusion

The program demonstrates the use of Python variables, arithmetic, conditions, loops and data structures to solve this case study using clearly defined classroom rules.

Testing and observations

For the sample values, march should be flagged as an unusual increase.

I can also test the program by changing a boundary value, such as a minimum mark, consumption slab, or budget. Trying zero or invalid values helps check how the conditions behave.

What I learned

This activity helped me practise Python variables, calculations, conditional statements, loops, and working with lists or dictionaries where needed.

Activity 3: Hospital Billing and Patient Classification

What the program does

Compute patient charges, concessions, payable amount and payment classification.

Rules used in this activity

Concession 20% for senior citizens age at least 60; 10% for low-income patients; only the larger concession applies; charges must be nonnegative; payment class: subsidized if concession applies, standard otherwise.

How I approached it

I added the three hospital charges, checked that none was negative, and applied the appropriate patient concession before classifying the payment.

Python program

```
patients = [
       ("Ravi", 35, False, 500, 1200, 2500),
       ("Meena", 67, False, 500, 1800, 3200),
       ("John", 42, True, 400, 800, 1500)
   ]
   for name, age, low_income, consultation, diagnostics, treatment in patients:
       charges = [consultation, diagnostics, treatment]
       if min(charges) < 0:
           print(name, "Invalid charges")
           continue
       gross = sum(charges)
       discount = 0.20 if age >= 60 else (0.10 if low_income else 0)
       payable = gross * (1 - discount)
       category = "Subsidized" if discount > 0 else "Standard"
       print(name, category, "Payable: Rs", round(payable, 2))
```

Activity 3: Results and observations

Sample execution

```
Terminal - case_03.py

$ python programs/case_03.py
Ravi Standard Payable: Rs 4200
Meena Subsidized Payable: Rs 4400.0
John Subsidized Payable: Rs 2430.0
```

Conclusion

The program demonstrates the use of Python variables, arithmetic, conditions, loops and data structures to solve this case study using clearly defined classroom rules.

Testing and observations

For the sample values, senior citizen receives 20% concession; low-income patient receives 10%.

I can also test the program by changing a boundary value, such as a minimum mark, consumption slab, or budget. Trying zero or invalid values helps check how the conditions behave.

What I learned

This activity helped me practise Python variables, calculations, conditional statements, loops, and working with lists or dictionaries where needed.

Activity 4: Employee Payroll and Deduction Analysis

What the program does

Calculate employee gross salary, progressive deduction and net pay.

Rules used in this activity

HRA 20% of basic, DA 10%, overtime Rs 200/hour; deductions 0% on first Rs 20000, 5% on next Rs 20000, 10% on remaining gross.

How I approached it

I calculated allowances, overtime, and incentives to get gross pay. Next, I applied the deduction slabs and found the net salary.

Python program

```
employees = [("Anita", 25000, 8, 3000), ("Vikram", 40000, 12, 5000)]
   for name, basic, overtime_hours, incentive in employees:
       hra = basic * 0.20
       da = basic * 0.10
       overtime = overtime_hours * 200
       gross = basic + hra + da + overtime + incentive
       deduction = min(max(gross - 20000, 0), 20000) * 0.05
       deduction += max(gross - 40000, 0) * 0.10
       net = gross - deduction
       print("EMPLOYEE:", name)
       print("Gross: Rs", round(gross, 2), "Deduction: Rs", round(deduction, 2), "Net: Rs",
 round(net, 2))
```

Activity 4: Results and observations

Sample execution



Conclusion

The program demonstrates the use of Python variables, arithmetic, conditions, loops and data structures to solve this case study using clearly defined classroom rules.

Testing and observations

For the sample values, net salary equals gross minus progressive deduction.

I can also test the program by changing a boundary value, such as a minimum mark, consumption slab, or budget. Trying zero or invalid values helps check how the conditions behave.

What I learned

This activity helped me practise Python variables, calculations, conditional statements, loops, and working with lists or dictionaries where needed.

Activity 5: Multi-Plan Telecommunications Cost Optimization

What the program does

Compare mobile plans by usage, validity and budget, including overage fees.

Rules used in this activity

Plans include data allowance in GB, calls in minutes, validity in days; excess data Rs 15/GB, excess calls Rs 1/min; usage: 25 GB, 450 min, 28 days; budget Rs 500.

How I approached it

I checked plan validity first, added any extra data and call charges, and selected the lowest-cost plan that stayed within the budget.

Python program

```
plans = [
       ("Basic", 199, 15, 300, 28),
       ("Smart", 299, 25, 500, 28),
       ("Plus", 399, 40, 1000, 30)
   ]
   data_used, calls_used, days_needed, budget = 25, 450, 28, 500
   choices = []
   for name, price, data, calls, validity in plans:
       if validity < days_needed:
           continue
       total = price + max(data_used - data, 0) * 15 + max(calls_used - calls, 0) * 1
       print(name, "Total: Rs", total)
       if total <= budget:
           choices.append((total, name))
   if choices:
       best = min(choices)
       print("Best eligible plan:", best[1], "Rs", best[0])
   else:
       print("No plan meets the budget")
```

Activity 5: Results and observations

Sample execution

```
Terminal - case_05.py

$ python programs/case_05.py
Basic Total: Rs 499
Smart Total: Rs 299
Plus Total: Rs 399
Best eligible plan: Smart Rs 299
```

Conclusion

The program demonstrates the use of Python variables, arithmetic, conditions, loops and data structures to solve this case study using clearly defined classroom rules.

Testing and observations

For the sample values, smart should be selected as the lowest-cost eligible plan.

I can also test the program by changing a boundary value, such as a minimum mark, consumption slab, or budget. Trying zero or invalid values helps check how the conditions behave.

What I learned

This activity helped me practise Python variables, calculations, conditional statements, loops, and working with lists or dictionaries where needed.

Activity 6: Railway Fare and Reservation Calculator

What the program does

Calculate fare for multiple passengers with distance slabs, travel class, concessions, reservation and cancellation fees.

Rules used in this activity

Base per km: first 100 km Rs 1, next 200 Rs 0.8, beyond Rs 0.6; sleeper multiplier 1.5, AC 2.5; senior age at least 60 gets 20% base fare concession; reservation Rs 30 per passenger; cancellation 15% of total.

How I approached it

I calculated the fare for the distance slabs, adjusted it for travel class and age, then added the reservation charge and cancellation penalty.

Python program

```
passengers = [("Ravi", 32, "Sleeper"), ("Lakshmi", 65, "AC")]
   distance = 350
   base = min(distance, 100) * 1
   base += min(max(distance - 100, 0), 200) * 0.8
   base += max(distance - 300, 0) * 0.6
   grand_total = 0
   for name, age, travel_class in passengers:
       multiplier = 2.5 if travel_class == "AC" else 1.5
       fare = base * multiplier
       if age >= 60:
           fare *= 0.80
       fare += 30
       grand_total += fare
       print(name, travel_class, "Fare: Rs", round(fare, 2))
   print("Total fare: Rs", round(grand_total, 2))
   print("Cancellation penalty: Rs", round(grand_total * 0.15, 2))
```

Activity 6: Results and observations

Sample execution

```
Terminal - case_06.py

$ python programs/case_06.py
Ravi Sleeper Fare: Rs 465.0
Lakshmi AC Fare: Rs 610.0
Total fare: Rs 1075.0
Cancellation penalty: Rs 161.25
```

Conclusion

The program demonstrates the use of Python variables, arithmetic, conditions, loops and data structures to solve this case study using clearly defined classroom rules.

Testing and observations

For the sample values, senior passenger fare includes 20% concession before reservation fee.

I can also test the program by changing a boundary value, such as a minimum mark, consumption slab, or budget. Trying zero or invalid values helps check how the conditions behave.

What I learned

This activity helped me practise Python variables, calculations, conditional statements, loops, and working with lists or dictionaries where needed.

Activity 7: Retail Billing with Multiple Discount Rules

What the program does

Generate itemized invoice with category tax, threshold discount and concession cap.

Rules used in this activity

Tax: grocery 5%, electronics 18%, stationery 12%; discount 10% of pretax subtotal when subtotal at least Rs 3000, capped at Rs 500; discount is apportioned before tax.

How I approached it

I found each item's amount, calculated the capped discount, distributed it across items, and applied the tax for each category.

Python program

```
items = [("Rice", "grocery", 2, 600), ("Headphones", "electronics", 1, 2200), ("Notebook",
"stationery", 3,
   100)]
   taxes = {"grocery":0.05, "electronics":0.18, "stationery":0.12}
   subtotal = sum(quantity * price for name, category, quantity, price in items)
   discount = min(subtotal * 0.10, 500) if subtotal >= 3000 else 0
   total_tax = 0
   for name, category, quantity, price in items:
       line_amount = quantity * price
       line_discount = discount * line_amount / subtotal
       tax = (line_amount - line_discount) * taxes[category]
       total_tax += tax
       print(name, "Amount:", line_amount, "Tax:", round(tax, 2))
   print("Subtotal:", subtotal, "Discount:", round(discount, 2))
   print("Tax:", round(total_tax, 2), "Final: Rs", round(subtotal - discount + total_tax,
2))
```

Activity 7: Results and observations

Sample execution

```
Terminal - case_07.py

$ python programs/case_07.py
Rice Amount: 1200 Tax: 54.0
Headphones Amount: 2200 Tax: 356.4
Notebook Amount: 300 Tax: 32.4
Subtotal: 3700 Discount: 370.0
Tax: 442.8 Final: Rs 3772.8
```

Conclusion

The program demonstrates the use of Python variables, arithmetic, conditions, loops and data structures to solve

Testing and observations

For the sample values, total discount is capped at Rs 500 and category taxes are applied.

I can also test the program by changing a boundary value, such as a minimum mark, consumption slab, or budget. Trying zero or invalid values helps check how the conditions behave.

What I learned

This activity helped me practise Python variables, calculations, conditional statements, loops, and working with lists or dictionaries where needed.

Activity 8: Manufacturing Cost and Profitability Analysis

What the program does

Compute manufacturing cost and compare expected profit against a target margin.

Rules used in this activity

100 units; material Rs 200/unit, labour Rs 80/unit, wastage 5% of material, machine cost Rs 4000; selling price Rs 380/unit; target profit margin 15% of sales.

How I approached it

I combined material, labour, wastage, and machine costs, then compared revenue and profit margin with the target.

Python program

```
quantity = 100
   material_per_unit = 200
   labour_per_unit = 80
   wastage_rate = 0.05
   machine_cost = 4000
   selling_price = 380
   target_margin = 0.15
   material = quantity * material_per_unit
   labour = quantity * labour_per_unit
   wastage = material * wastage_rate
   cost = material + labour + wastage + machine_cost
   revenue = quantity * selling_price
   profit = revenue - cost
   margin = profit / revenue if revenue > 0 else 0
   print("Cost: Rs", cost, "Revenue: Rs", revenue)
   print("Profit: Rs", profit, "Margin:", round(margin * 100, 2), "%")
   print("Target achieved:", margin >= target_margin)
```

Activity 8: Results and observations

Sample execution

```
Terminal - case_08.py

$ python programs/case_08.py
Cost: Rs 33000.0 Revenue: Rs 38000
Profit: Rs 5000.0 Margin: 13.16 %
Target achieved: False
```

Conclusion

The program demonstrates the use of Python variables, arithmetic, conditions, loops and data structures to solve this case study using clearly defined classroom rules.

Testing and observations

For the sample values, the output should identify whether the 15% target margin is achieved.

I can also test the program by changing a boundary value, such as a minimum mark, consumption slab, or budget. Trying zero or invalid values helps check how the conditions behave.

What I learned

This activity helped me practise Python variables, calculations, conditional statements, loops, and working with lists or dictionaries where needed.

Activity 9: Multi-Currency Travel Budget Analysis

What the program does

Convert expenses into INR with supplied rates and conversion fee, compare with travel budget and identify largest expense category.

Rules used in this activity

Exchange rates supplied: USD Rs 84, EUR Rs 92, INR Rs 1; conversion fee 2% on non-INR expenses; overall budget Rs 50000.

How I approached it

I converted each expense to rupees, included conversion fees where applicable, and compared the total with the travel budget.

Python program

```
rates = {"USD":84, "EUR":92, "INR":1}
   expenses = [("Flights", 250, "USD"), ("Hotel", 180, "EUR"), ("Food", 6000, "INR"),
("Transport", 45, "USD")]
   budget = 50000
   totals = {}
   for category, amount, currency in expenses:
       converted = amount * rates[currency]
       if currency != "INR":
           converted *= 1.02
       totals[category] = totals.get(category, 0) + converted
       print(category, "Rs", round(converted, 2))
   total = sum(totals.values())
   print("Total: Rs", round(total, 2), "Within budget:", total <= budget)
   print("Largest category:", max(totals, key=totals.get))
```

Activity 9: Results and observations

Sample execution

```
Terminal - case_09.py

$ python programs/case_09.py
Flights Rs 21420.0
Hotel Rs 16891.2
Food Rs 6000
Transport Rs 3855.6
Total: Rs 48166.8 Within budget: True
Largest category: Flights
```

Conclusion

Testing and observations

For the sample values, the program should report the highest-cost travel category.

I can also test the program by changing a boundary value, such as a minimum mark, consumption slab, or budget. Trying zero or invalid values helps check how the conditions behave.

What I learned

This activity helped me practise Python variables, calculations, conditional statements, loops, and working with lists or dictionaries where needed.

Activity 10: Campus Event Financial Planning

What the program does

Estimate event costs, sponsorship shortfall and registrations needed to break even.

Rules used in this activity

Venue Rs 10000, equipment Rs 6000, catering Rs 120 per attendee, transport Rs 4000, certificates Rs 30 per attendee; registration Rs 250; sponsorship Rs 12000; planned 150 attendees.

How I approached it

I separated fixed and per-person event costs, calculated expected income, and found the minimum registrations needed to break even.

Python program

```
venue, equipment, transport = 10000, 6000, 4000
   catering_per_person, certificate_per_person = 120, 30
   fee, sponsorship, expected = 250, 12000, 150
   fixed_cost = venue + equipment + transport
   variable_cost = catering_per_person + certificate_per_person
   total_cost = fixed_cost + variable_cost * expected
   registration_income = fee * expected
   shortfall = max(total_cost - registration_income - sponsorship, 0)
   contribution = fee - variable_cost
   minimum = 0
   if contribution > 0:
       while sponsorship + contribution * minimum < fixed_cost:
           minimum += 1
       print("Minimum break-even registrations:", minimum)
   else:
       print("Break-even impossible with current fee")
   print("Estimated cost: Rs", total_cost)
   print("Registration income: Rs", registration_income)
   print("Sponsorship shortfall: Rs", shortfall)
```

Activity 10: Results and observations

Sample execution



Conclusion

The program demonstrates the use of Python variables, arithmetic, conditions, loops and data structures to solve this case study using clearly defined classroom rules.

Testing and observations

For the sample values, break-even registrations account for both fixed and per-attendee costs.

I can also test the program by changing a boundary value, such as a minimum mark, consumption slab, or budget. Trying zero or invalid values helps check how the conditions behave.

What I learned

This activity helped me practise Python variables, calculations, conditional statements, loops, and working with lists or dictionaries where needed.